Thesis Progress Update: Global Complementary-Model Routing

Eren Olmez — M.Sc. Thesis — 13 October 2026

Results saved through 12 October 2026; update to the previous 1 October report.

1. Model deployment and static measurements

Since the previous report, the candidate pool has grown from one ResNet-18 pruning family to eight architectures and 213 deployment configurations. PC static measurements are complete, as is the first 213-output correctness-router experiment. Some routed configurations reduced measured latency and energy relative to an accuracy-matched static model, although the improvement did not hold consistently across seeds. The Jetson study remains unfinished.

Stage	Saved progress	Interpretation
FP32 source models / ONNX export	71/71	8 baselines + 63 pruning variants
Diagnostic precision accuracy	213/213	Completed; heterogeneous runtimes
PC TensorRT static targets	213/213	6,390 timing/energy repetitions
Within-architecture scalar controls	1,941 routes	Stopped; not global progress
Global scalar routing	52/1,440	Stopped for correctness-method switch
213-output correctness baseline	120/120	Complete; 53 accuracy-compliant
Jetson architecture engines / accuracy	75/213	Paused, target-native builds
Jetson fully measured static targets	74/213	2,232/6,390 saved repetitions

All 71 ONNX sources passed PyTorch/ONNX comparisons at batches 1, 8 and 128: 213 checks in 67.31 s. The maximum absolute logit difference was 0.000280, and top-1 predictions agreed on the checked sample. The source graphs occupy 1.70 GB. INT8 calibration used a fixed set of 1,024 training images. Each TensorRT target was evaluated on 5,000 validation and 10,000 test images, with five static repetitions at batches 1/8/16/32/64/128.

The table reports TensorRT test accuracy for the unpruned models. FP16 latency measures normalized host input to output logits *per batch of 32*; GPU energy is in mJ/image. Timing and energy values are means of five repeats. This static timing excludes RGB normalization and routing, so it should not be confused with the full routed measurement or with Jetson results.

Architecture	FP32%	FP16%	INT8%	FP16 ms/b32	mJ/image	Fit min[a]
ResNet-18	79.16	79.17	77.66	9.131	8.830	90.14
ResNet-34	81.82	81.85	73.41	7.451	10.055	156.38
ResNet-50	84.39	84.38	79.69	25.882	25.075	348.67
ResNet-101	85.80	85.85	66.80	42.447	40.749	216.95
MobileNetV3-Small	62.38	62.36	15.50	2.709	2.904	24.72
MobileNetV3-Large	81.34	81.32	30.77	4.053	5.143	103.31
EfficientNet-B0	82.73	82.71	13.59	6.975	9.231	196.34
ShuffleNetV2 x1.0	75.64	75.66	74.66	7.597	7.637	29.12

[a] Original classifier baseline training durations, not router training or precision conversion. Classifier checkpoints are retained rather than retrained for each precision.

FP16 generally preserved accuracy. The earlier diagnostic precision study, which used different runtimes for different precisions, found mean losses of 0.08 ppfor FP16 and 21.08 ppfor INT8. These describe accuracy, not comparable GPU speedups. EfficientNet-B0 p0 lost 69.16 ppin that diagnostic INT8 test; its TensorRT INT8 accuracy was also poor, at 13.59%. Engine inspection confirmed integer compute: p0 INT8 integer-layer counts were 16/26/54/105 for ResNet-18/34/50/101, 54/64 for MobileNet Small/Large, 82 for EfficientNet and 57 for ShuffleNet. Thus, successful INT8 execution did not necessarily produce a usable classifier.

2. Earlier routing tests and the change in method

Screening was a preliminary accuracy check, not a learning method. Each pruned model was compared with its own unpruned model (p0) to see whether its accuracy loss stayed below 0.5, 1, 1.5 or 2 percentage points. For example, MobileNetV3-Small p10 scored 60.82% against p0's 62.44%: a loss of 1.62 points, so it passed the 2-point check but not the 1.5-point check. Its MAC count was 27.11% lower. No pruned ResNet, EfficientNet or MobileNet-Large passed even the 2-point check. This left nine source models, or 27 precision configurations.

That shortlist was abandoned as a restriction on routing. A model can have lower overall accuracy yet correctly classify images that another model gets wrong. All 213 configurations therefore remain selectable. The original screening used historical test accuracy and is only descriptive; current policy selection uses validation data. The rejected pruning and quantization results remain recorded.

The earlier scalar routers predicted a single score derived from the cheapest correct candidate's position in a cost ordering. Methods included a gradient stump, depth-3 tree, logistic regression, depth-2 XGBoost and TinyCNN. Image-tree scores were combined using the mean or percentiles 10/25/50/70/80/90. The logistic target was a binary rank-related target, rather than a separate correctness estimate for every candidate.

Within-architecture control	Saved	Accuracy pass	Positive latency[b]	Positive energy[b]
Gradient stump	180	180	48	42
Batch tree d3	161	152	71	64
Batch logistic	160	103	2	1
Batch XGBoost d2	160	120	59	53
Batch TinyCNN	160	85	1	1
Image tree: mean	160	100	49	44
Image tree: p10	160	128	44	42
Image tree: p25	160	122	50	47
Image tree: p50	160	153	68	62
Image tree: p70	160	147	69	61
Image tree: p80	160	151	65	59
Image tree: p90	160	157	68	60
Total	1,941	1,598	594	536

[b] Positive savings among accuracy-compliant rows against the recorded eligible static comparator. Rows pool architectures, budgets, objectives and seeds; cached comparators and timing variation make these *control observations*, not confirmed independent wins.

The within-architecture run stopped after 1,941 configurations because it could not test selection across architectures. These results remain controls. The subsequent global scalar run allowed all 213 choices, kept at most two classifier engines resident, and measured fresh static comparators.

Global scalar method	Saved	Accuracy pass	Positive latency[c]	Positive energy[c]
Gradient stump	20	20	2	2
Batch tree d3	20	20	2	2
Batch logistic	12	5	0	0
Total	52	45	4	4

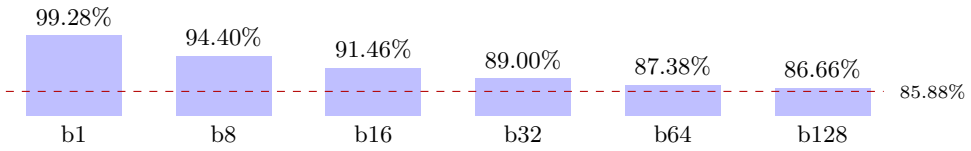
[c] Same count definition as above; fresh but sequential comparators. XGBoost, TinyCNN and image aggregates had no saved global measured routes at the priority switch. The 52 rows are only 3.61% of the originally planned 1,440-case global matrix.

The global scalar run was stopped after 52 complete configurations to test per-model correctness instead. It was not fully evaluated. Some apparent gains also came from timing variation: one batch-64 policy selected ResNet-101 FP16 for every image, but routed passes took about 22.00/22.04/15.21/14.58/14.59 s, compared with preceding same-model static passes of 21.26/21.24/21.32/21.28/21.34 s. This cannot demonstrate a benefit from model selection. A later same-model comparison used 3.21% more routed energy. Alternating static and routed measurements is needed to investigate this variation.

3. Model complementarity and correctness-router training

For each seed (42/45/48), the 5,000 classifier-validation images were divided into 3,000 router-training and 2,000 policy-calibration images. "Validation-only" refers to the source of this supervision: the router was trained, but no held-out test labels were used to fit its weights or choose its policy. The 10,000 test images were used for final evaluation. One limitation is that the same 5,000-image validation set had already informed classifier checkpoint selection.

The validation-selected reference, ResNet-101 p0 FP32, obtained 85.88%. An oracle that knows the correct label and can choose per image obtained 99.28% over all 213 candidates: only 36 of 5,000 images were wrong for every model. Restricting this oracle to the eight unpruned FP32 models gave 95.68%. These are upper bounds, not measured learned-router performance. MobileNet-Large p0 FP16 corrected 250 reference errors but made 441 errors on images the reference got right; EfficientNet p0 FP16 corrected 246 and introduced 383. Their lower total accuracy therefore does not imply that their predictions are redundant.



Oracle accuracy when one model is selected per batch, using one fixed validation shuffle. Bars start at 80%; the dashed line marks the reference. At batch 128, the possible accuracy improvement is only 0.78 pp, compared with 13.40 ppfor per-image selection.

The router uses a 16×16 image thumbnail, a small depthwise CNN and 213 output logits. Each output learns whether its corresponding classifier is correct, using binary cross-entropy; several outputs can be correct on the same image. Training used 20 fixed epochs and Adam with learning rate 0.001. For batch B, the selected model is

$$q_m(B) = |B|^{-1} \sum_{x \in B} \sigma(z_m(x)), \qquad m^*(B) = \arg \max_{m \in \{1,\ldots,213\}} \left[q_m(B) - \lambda \frac{C_m(B)}{\max_j C_j(B)} \right].$$

Here $q_m(B)$ is the mean predicted correctness of model m over the batch, and $C_m(B)$ is its measured static cost. The penalty λ is calibrated separately for latency and energy under allowances of 0.5/1/1.5/2 pp. The accuracy constraint applies to the resulting policy, not to every model individually. A lower-accuracy model can therefore be selected for a batch if the router predicts that it will perform well on those inputs.

Three router fits and six objective/seed calibration groups completed. Each seed was exported to six fixed-shape TensorRT engines, including the true final batch of 16. Maximum PyTorch/TensorRT output differences were 2.3842×10^{-7} for seeds 42/45 and 1.7881×10^{-7} for seed 48. The times below cover fitting, export and calibration, not classifier training. The reduction in fitting loss does not, on its own, establish test accuracy or runtime savings.

Seed	Stage seconds	Epoch-1 fitting BCE	Epoch-20 fitting BCE
42	53.72	0.65984	0.45293
45	46.00	0.63925	0.44929
48	42.52	0.65903	0.45325

Per-model correctness supervision was already present in the earlier 10-candidate benchmark. The current experiment extends it to 213 architecture/precision/pruning choices with measured cache and inference costs. PERTINENCE [1] and Taylor et al. [2] are closely related input-based model-selection studies. DynamicDet [3] motivates defining difficulty through the benefit of additional computation; CARN [4] and adaptive-inference policies [5] provide other relevant comparisons.

4. Complete correctness matrix: all 120 saved tests

Each cell is **accuracy loss (pp) / latency saving (%) / GPU-energy saving (%)** for one seed. Savings are against that row's frozen validation-eligible, freshly measured static comparator; negative values mean slower/more energy. A superscript $*$ marks a held-out accuracy-budget failure, even if savings are positive. Five full 10,000-image repeats per cell. Results are descriptive, never used to retune the policy.

Objective	Batch	Budget	Seed 42: loss / L / E	Seed 45: loss / L / E	Seed 48: loss / L / E
Latency	8	0.5	0.26 / -143.7 / -150.8	0.33 / +0.6 / +1.1	1.50 / -20.2 / -22.7*
	8	1	0.75 / -513.9 / -430.5	1.51 / -36.5 / -39.1*	1.83 / -19.2 / -21.9*
	8	1.5	1.33 / -712.9 / -472.7	1.71 / -55.0 / -58.9*	2.59 / -198.2 / -210.3*
	8	2	1.83 / -713.7 / -350.7	2.15 / -94.2 / -101.9*	3.10 / -222.9 / -240.3*
	16	0.5	0.28 / -122.9 / -94.1	1.09 / +11.5 / +11.6*	0.28 / -0.6 / -0.7
	16	1	0.88 / -191.7 / -129.1	1.52 / +18.4 / +18.5*	0.81 / +6.2 / +5.9
	16	1.5	2.00 / -222.4 / -151.0*	1.89 / +23.4 / +23.7*	2.03 / -18.6 / -18.4*
	16	2	2.82 / -85.1 / -50.2*	2.40 / +30.6 / +30.5*	2.03 / -18.7 / -19.6*
	32	0.5	0.45 / +0.7 / +0.4	0.71 / +4.3 / +4.4*	0.85 / +12.9 / +13.5*
	32	1	0.84 / +6.0 / +5.4	0.79 / +7.2 / +6.8	1.42 / +21.6 / +21.2*
	32	1.5	1.38 / +16.6 / +16.3	1.05 / +10.3 / +9.8	1.42 / -18.3 / -18.3
	32	2	2.30 / +58.5 / +57.9*	1.57 / +18.3 / +17.6	2.31 / -5.8 / -6.3*
	64	0.5	0.87 / +13.2 / +13.0*	0.28 / +2.5 / +2.5	0.39 / +4.6 / +4.5
	64	1	1.78 / +26.7 / +26.2*	0.63 / +6.8 / +6.9	0.96 / +11.4 / +11.1
	64	1.5	1.78 / +26.7 / +26.6*	1.22 / +11.4 / +10.9	0.96 / -45.7 / -44.9
	64	2	2.72 / +43.7 / +43.2*	2.57 / +28.3 / +27.1*	3.94 / +45.4 / +45.0*
	128	0.5	0.57 / +7.8 / +7.6*	0.87 / +11.8 / +11.5*	0.61 / +9.0 / +8.6*
	128	1	1.78 / +32.2 / +32.4*	0.87 / +11.8 / +11.1	0.86 / +16.0 / +15.8
	128	1.5	1.78 / +32.1 / +32.5*	1.87 / +27.1 / +26.2*	0.86 / -47.7 / -48.2
	128	2	1.78 / +32.0 / +32.6	1.87 / +26.9 / +25.5	3.55 / +24.5 / +24.6*
Energy	8	0.5	0.28 / -109.5 / -117.8	0.28 / +0.4 / +0.3	0.77 / -118.8 / -125.5*
	8	1	0.68 / -332.4 / -329.1	1.47 / -1.6 / -3.5*	1.66 / -286.9 / -290.7*
	8	1.5	1.07 / -545.2 / -418.9	1.86 / -6.8 / -9.4*	3.10 / -1053.0 / -959.5*
	8	2	1.43 / -952.4 / -573.4	2.30 / -9.6 / -12.5*	3.44 / -1304.0 / -857.2*
	16	0.5	0.30 / -180.0 / -150.6	1.10 / +11.8 / +11.6*	0.25 / -4.9 / -2.8
	16	1	0.85 / -279.8 / -212.2	1.55 / +18.7 / +18.0*	0.79 / +6.5 / +5.6
	16	1.5	1.77 / -392.1 / -277.2*	1.90 / +23.7 / +23.8*	1.96 / -51.2 / -53.9*
	16	2	2.78 / -404.9 / -280.8*	2.42 / +30.6 / +29.7*	3.01 / -42.2 / -36.3*
	32	0.5	0.67 / -33.1 / -14.5*	0.75 / +4.7 / +4.8*	1.27 / +16.7 / +16.4*
	32	1	0.67 / -8.3 / -10.7	0.79 / +7.1 / +7.5	1.27 / +19.1 / +19.1*
	32	1.5	1.68 / +1.7 / -0.1*	1.05 / +10.4 / +10.4	1.27 / -21.8 / -28.4
	32	2	2.17 / +13.4 / +12.7*	1.61 / +18.5 / +17.9	1.97 / -8.2 / -9.5
	64	0.5	0.57 / +8.3 / +8.0*	0.53 / +5.5 / +5.3*	0.16 / +2.0 / +0.0
	64	1	1.36 / +19.6 / +19.6*	1.06 / +9.5 / +9.4*	0.62 / +39.1 / +33.5
	64	1.5	1.36 / +19.6 / +19.7	1.46 / +15.5 / +15.6	1.14 / -40.1 / -40.4
	64	2	2.29 / +13.8 / +13.0*	2.31 / +29.1 / +28.2*	2.15 / -15.4 / -14.9*
	128	0.5	0.41 / +6.2 / +5.9	1.29 / +19.2 / +18.7*	0.41 / +5.4 / +5.1
	128	1	1.29 / +15.6 / +15.2*	1.29 / +23.9 / +23.2*	0.68 / +14.2 / +14.5
	128	1.5	1.29 / +24.9 / +24.6	1.87 / +27.2 / +25.7*	0.68 / -50.3 / -50.2
	128	2	2.72 / +45.6 / +45.6*	1.87 / +27.2 / +26.4	3.51 / +17.8 / +18.2*

A larger allowance can reduce savings even when routed accuracy is unchanged, because it permits comparison with a cheaper static model. Savings marked $*$ do not meet the stated accuracy requirement. A rounded +0.0% is a very small recorded positive value, not evidence of a practically useful benefit.

Across the 120 tests, 53 met the accuracy allowance; 31 of these saved latency and 31 saved energy. By batch size 8/16/32/64/128, accuracy-pass counts were 10/8/13/11/11 out of 24 tests each. The corresponding counts with both an accuracy pass and positive latency savings were 2/2/9/9/9; energy counts were also 2/2/9/9/9. Counts include different objectives, allowances and seeds, so they are not independent discoveries.

5. Discussion and remaining work

The $3 \times 2 \times 5 \times 4$ matrix required 600 full routed test passes, or 6 million image evaluations, plus fresh static comparisons. Of 120 configurations, 53 (44.17%) met the accuracy allowance. The batch-128 examples below show both improvements and failures. Latency is wall-time ms/image and energy is GPU mJ/image, averaged over five repeats. The static comparator is ResNet-101 FP16 except in rows 4 and 6, where it is ResNet-50 FP16.

Objective/seed	Budget	Acc.%	Loss pp	ms/img	mJ/img	L saving%	E saving%
Latency/42	2.0	84.02	1.78	1.035	31.936	32.05	32.56
Latency/45	1.0	84.93	0.87	1.344	41.893	11.83	11.10
Latency/48	1.0	84.94	0.86	1.279	39.718	16.00	15.77
Latency/48	1.5	84.94	0.86	1.277	39.529	-47.66	-48.21
Energy/42	1.5	84.51	1.29	1.144	35.517	24.92	24.64
Latency/48	2.0	82.25	3.55*	0.653	20.376	24.49	24.61

In latency/42/b128/2 pp, the router selected R101 FP16 for 5,136 images and R34 FP16 for 4,864. The corresponding 1 ppsettings selected R101/EfficientNet for 7,040/2,960 images (seed 45), and R101/R34 for 7,424/2,576 (seed 48). These examples used two loads and no evictions per repeat. All 213 models were available; the concentration on two models was the policy's outcome, not an imposed shortlist. However, 67 tests failed the accuracy allowance, and no latency setting both passed and improved on its comparator for all three seeds.

Timing includes RGB transfer, GPU normalization, the router, selection, engine loading/eviction, inference and synchronization. Decode/resize and batch pinning are excluded equally from both paths. The example routers spent about 1.30–1.34 ms/batch on the GPU decision component. Training took seconds; five full passes and fresh static comparisons per configuration accounted for much of the testing time. NVML measures GPU energy, including idle/display load, not whole-PC energy. Sequential timing and temperature/load changes remain possible sources of variation, so alternating static/routed tests is a priority.

Jetson. The study is paused with 75/213 engines built and accuracy-validated, 74/213 fully measured static targets, and 2,232/6,390 energy repetitions. EfficientNet and MobileNet-Large account for 60 targets/1,800 repeats; MobileNet-Small p40 INT8 has 12/30 repeats saved, ending at batch 16, repeat 1. Three architectures are staged; five still need transfer. The last saved target reached 54.375°C. No new global Jetson routing has run. Detailed target results remain on the paused device; the local update uses its verified checkpoint counts. Engines must be built with Jetson TensorRT 8.6.2, not reused from the PC.

Next experiments. The eight classifiers used the same 45,000 training and 5,000 validation images, with no unused reserved pool. More router supervision would require either explicitly exploratory predictions on classifier-training images, or slower classifier retraining/out-of-fold predictions. This choice is still pending. Seven additional correctness routers are prepared: depth-1/depth-3 trees, logistic heads, depth-2 XGBoost (32 trees per candidate), and TinyCNN inputs 8/16/32. The current smaller CNN remains a separate baseline. Eleven CPU tests passed in 15.699 s; real-data GPU fitting, export and runtime evaluation are unfinished.

The proposed first comparison uses seed 42, batches 32/128, latency/energy objectives and 1/2 ppallowances, with one full-test repeat: 56 configurations, or 64 with a newly trained current-CNN baseline. Full confirmation would add multiple seeds and five repeats, giving 840 additional tests (960 including the existing 120). No parameters will be selected from the completed test results. Per-configuration numbers and source hashes remain in `numerical_summary.json`; full repetitions are retained locally in `numerical_results.json`. Automation is paused, and this revision did not restart any experiments.

References

[1] O. Shende, G. Ananthanarayanan and M. Traiola, "PERTINENCE: Input-Based Opportunistic Neural Network Dynamic Execution," *IEEE Access*, 2026. doi:10.1109/ACCESS.2026.3707342.

[2] B. Taylor et al., "Adaptive Deep Learning Model Selection on Embedded Systems," *LCTES*, 2018. doi:10.1145/3211332.3211336.

[3] Z. Lin et al., "DynamicDet: A Unified Dynamic Architecture for Object Detection," *CVPR*, 2023.

[4] R. Gaire and A. Roohi, "CARN: Complexity-Aware Routing Network for Efficient and Adaptive Inference," *CVPR Workshop*, 2025.

[5] T. Bolukbasi et al., "Adaptive Neural Networks for Efficient Inference," *ICML*, PMLR 70, 2017.